

CONTENTS

1	Whetting Your Appetite	3
2	Using the Python Interpreter	5
2.1	Invoking the Interpreter	5
2.1.1	Argument Passing	6
2.1.2	Interactive Mode	6
2.2	The Interpreter and Its Environment	6
2.2.1	Source Code Encoding	6
3	An Informal Introduction to Python	9
3.1	Using Python as a Calculator	9
3.1.1	Numbers	9
3.1.2	Strings	11
3.1.3	Lists	14
3.2	First Steps Towards Programming	15
4	More Control Flow Tools	17
4.1	if Statements	17
4.2	for Statements	17
4.3	The range() Function	18
4.4	break and continue Statements, and else Clauses on Loops	19
4.5	pass Statements	20
4.6	Defining Functions	20
4.7	More on Defining Functions	22
4.7.1	Default Argument Values	22
4.7.2	Keyword Arguments	23
4.7.3	Special parameters	24
4.7.4	Arbitrary Argument Lists	27
4.7.5	Unpacking Argument Lists	27
4.7.6	Lambda Expressions	27
4.7.7	Documentation Strings	28
4.7.8	Function Annotations	28
4.8	Intermezzo: Coding Style	29
5	Data Structures	31
5.1	More on Lists	31
5.1.1	Using Lists as Stacks	32
5.1.2	Using Lists as Queues	33
5.1.3	List Comprehensions	33
5.1.4	Nested List Comprehensions	34
5.2	The del statement	35
5.3	Tuples and Sequences	36
5.4	Sets	37
5.5	Dictionaries	37
5.6	Looping Techniques	38

5.7	More on Conditions	40
5.8	Comparing Sequences and Other Types	40
6	Modules	41
6.1	More on Modules	42
6.1.1	Executing modules as scripts	43
6.1.2	The Module Search Path	43
6.1.3	“Compiled” Python files	44
6.2	Standard Modules	44
6.3	The <code>dir()</code> Function	45
6.4	Packages	46
6.4.1	Importing <code>*</code> From a Package	47
6.4.2	Intra-package References	48
6.4.3	Packages in Multiple Directories	48
7	Input and Output	49
7.1	Fancier Output Formatting	49
7.1.1	Formatted String Literals	50
7.1.2	The String <code>format()</code> Method	51
7.1.3	Manual String Formatting	52
7.1.4	Old string formatting	52
7.2	Reading and Writing Files	53
7.2.1	Methods of File Objects	54
7.2.2	Saving structured data with <code>json</code>	55
8	Errors and Exceptions	57
8.1	Syntax Errors	57
8.2	Exceptions	57
8.3	Handling Exceptions	58
8.4	Raising Exceptions	60
8.5	Exception Chaining	61
8.6	User-defined Exceptions	61
8.7	Defining Clean-up Actions	62
8.8	Predefined Clean-up Actions	63
9	Classes	65
9.1	A Word About Names and Objects	65
9.2	Python Scopes and Namespaces	66
9.2.1	Scopes and Namespaces Example	67
9.3	A First Look at Classes	68
9.3.1	Class Definition Syntax	68
9.3.2	Class Objects	68
9.3.3	Instance Objects	69
9.3.4	Method Objects	69
9.3.5	Class and Instance Variables	70
9.4	Random Remarks	71
9.5	Inheritance	72
9.5.1	Multiple Inheritance	73
9.6	Private Variables	74
9.7	Odds and Ends	74
9.8	Iterators	75
9.9	Generators	76
9.10	Generator Expressions	77
10	Brief Tour of the Standard Library	79
10.1	Operating System Interface	79
10.2	File Wildcards	79
10.3	Command Line Arguments	80
10.4	Error Output Redirection and Program Termination	80

10.5	String Pattern Matching	80
10.6	Mathematics	81
10.7	Internet Access	81
10.8	Dates and Times	82
10.9	Data Compression	82
10.10	Performance Measurement	82
10.11	Quality Control	83
10.12	Batteries Included	83
11	Brief Tour of the Standard Library — Part II	85
11.1	Output Formatting	85
11.2	Templating	86
11.3	Working with Binary Data Record Layouts	87
11.4	Multi-threading	87
11.5	Logging	88
11.6	Weak References	88
11.7	Tools for Working with Lists	89
11.8	Decimal Floating Point Arithmetic	90
12	Virtual Environments and Packages	91
12.1	Introduction	91
12.2	Creating Virtual Environments	91
12.3	Managing Packages with pip	92
13	What Now?	95
14	Interactive Input Editing and History Substitution	97
14.1	Tab Completion and History Editing	97
14.2	Alternatives to the Interactive Interpreter	97
15	Floating Point Arithmetic: Issues and Limitations	99
15.1	Representation Error	101
16	Appendix	105
16.1	Interactive Mode	105
16.1.1	Error Handling	105
16.1.2	Executable Python Scripts	105
16.1.3	The Interactive Startup File	106
16.1.4	The Customization Modules	106
A	Glossary	107
B	About these documents	119
B.1	Contributors to the Python Documentation	119
C	History and License	121
C.1	History of the software	121
C.2	Terms and conditions for accessing or otherwise using Python	122
C.2.1	PSF LICENSE AGREEMENT FOR PYTHON 3.9.1	122
C.2.2	BEOPEN.COM LICENSE AGREEMENT FOR PYTHON 2.0	123
C.2.3	CNRI LICENSE AGREEMENT FOR PYTHON 1.6.1	124
C.2.4	CWI LICENSE AGREEMENT FOR PYTHON 0.9.0 THROUGH 1.2	125
C.2.5	ZERO-CLAUSE BSD LICENSE FOR CODE IN THE PYTHON 3.9.1 DOCUMENTA- TION	125
C.3	Licenses and Acknowledgements for Incorporated Software	126
C.3.1	Mersenne Twister	126
C.3.2	Sockets	127
C.3.3	Asynchronous socket services	127
C.3.4	Cookie management	128
C.3.5	Execution tracing	128

C.3.6	UUencode and UUdecode functions	129
C.3.7	XML Remote Procedure Calls	129
C.3.8	test_epoll	130
C.3.9	Select kqueue	130
C.3.10	SipHash24	131
C.3.11	strtod and dtoa	131
C.3.12	OpenSSL	132
C.3.13	expat	134
C.3.14	libffi	134
C.3.15	zlib	135
C.3.16	cfuhash	135
C.3.17	libmpdec	136
C.3.18	W3C C14N test suite	136
D	Copyright	139
	Index	141

Python is an easy to learn, powerful programming language. It has efficient high-level data structures and a simple but effective approach to object-oriented programming. Python's elegant syntax and dynamic typing, together with its interpreted nature, make it an ideal language for scripting and rapid application development in many areas on most platforms.

The Python interpreter and the extensive standard library are freely available in source or binary form for all major platforms from the Python Web site, <https://www.python.org/>, and may be freely distributed. The same site also contains distributions of and pointers to many free third party Python modules, programs and tools, and additional documentation.

The Python interpreter is easily extended with new functions and data types implemented in C or C++ (or other languages callable from C). Python is also suitable as an extension language for customizable applications.

This tutorial introduces the reader informally to the basic concepts and features of the Python language and system. It helps to have a Python interpreter handy for hands-on experience, but all examples are self-contained, so the tutorial can be read off-line as well.

For a description of standard objects and modules, see [library-index](#). [reference-index](#) gives a more formal definition of the language. To write extensions in C or C++, read [extending-index](#) and [c-api-index](#). There are also several books covering Python in depth.

This tutorial does not attempt to be comprehensive and cover every single feature, or even every commonly used feature. Instead, it introduces many of Python's most noteworthy features, and will give you a good idea of the language's flavor and style. After reading it, you will be able to read and write Python modules and programs, and you will be ready to learn more about the various Python library modules described in [library-index](#).

The [Glossary](#) is also worth going through.

WHETTING YOUR APPETITE

If you do much work on computers, eventually you find that there's some task you'd like to automate. For example, you may wish to perform a search-and-replace over a large number of text files, or rename and rearrange a bunch of photo files in a complicated way. Perhaps you'd like to write a small custom database, or a specialized GUI application, or a simple game.

If you're a professional software developer, you may have to work with several C/C++/Java libraries but find the usual write/compile/test/re-compile cycle is too slow. Perhaps you're writing a test suite for such a library and find writing the testing code a tedious task. Or maybe you've written a program that could use an extension language, and you don't want to design and implement a whole new language for your application.

Python is just the language for you.

You could write a Unix shell script or Windows batch files for some of these tasks, but shell scripts are best at moving around files and changing text data, not well-suited for GUI applications or games. You could write a C/C++/Java program, but it can take a lot of development time to get even a first-draft program. Python is simpler to use, available on Windows, Mac OS X, and Unix operating systems, and will help you get the job done more quickly.

Python is simple to use, but it is a real programming language, offering much more structure and support for large programs than shell scripts or batch files can offer. On the other hand, Python also offers much more error checking than C, and, being a *very-high-level language*, it has high-level data types built in, such as flexible arrays and dictionaries. Because of its more general data types Python is applicable to a much larger problem domain than Awk or even Perl, yet many things are at least as easy in Python as in those languages.

Python allows you to split your program into modules that can be reused in other Python programs. It comes with a large collection of standard modules that you can use as the basis of your programs — or as examples to start learning to program in Python. Some of these modules provide things like file I/O, system calls, sockets, and even interfaces to graphical user interface toolkits like Tk.

Python is an interpreted language, which can save you considerable time during program development because no compilation and linking is necessary. The interpreter can be used interactively, which makes it easy to experiment with features of the language, to write throw-away programs, or to test functions during bottom-up program development. It is also a handy desk calculator.

Python enables programs to be written compactly and readably. Programs written in Python are typically much shorter than equivalent C, C++, or Java programs, for several reasons:

- the high-level data types allow you to express complex operations in a single statement;
- statement grouping is done by indentation instead of beginning and ending brackets;
- no variable or argument declarations are necessary.

Python is *extensible*: if you know how to program in C it is easy to add a new built-in function or module to the interpreter, either to perform critical operations at maximum speed, or to link Python programs to libraries that may only be available in binary form (such as a vendor-specific graphics library). Once you are really hooked, you can link the Python interpreter into an application written in C and use it as an extension or command language for that application.

By the way, the language is named after the BBC show “Monty Python’s Flying Circus” and has nothing to do with reptiles. Making references to Monty Python skits in documentation is not only allowed, it is encouraged!